

Static Load Balancing for a CPU + GPU Stencil

Computación en GPU

Author: Matías Saavedra

Document: engineering report (hybrid load balancing)

Last updated: Saturday 4th July, 2026

Santiago de Chile

Resumen

The **HybridEngine** runs the CPU and the GPU together on one Game of Life board (High-Life, **B36/S23**), split by rows with Divisible Load Theory (DLT, Barlas **S11.3**). This report analyses a full benchmark sweep on a GTX 1660 Ti Max-Q paired with a 12-thread CPU. The hybrid is correct and the DLT split lands exactly where the theory predicts ($\sim 99.4\%$ GPU / 0.6% CPU), but the GPU is $\sim 160\times$ the CPU, so the load-balancing ceiling is a $\sim 0.6\%$ speed-up — smaller than the per-step coordination cost. The hybrid therefore does not beat pure GPU here. A raw reading of the sweep looks like it does at large N ; that turned out to be a confound — the hybrid runs a faster, branch-free GPU kernel than the standalone CUDA engine. Controlling for the kernel, load balancing is a net loss at small/mid N and break-even at large N . The real performance lever the exercise surfaced is the halo kernel, not the balancing. We also document an OpenCL driver regression seen during the run.

Índice de Contenidos

1. Scope and setup	1
2. Throughput results	1
3. The load-balancing split (DLT S11.3)	2
4. The figure	3
5. Unexpected findings	3
5.1. “Hybrid beats CUDA at large N ” is a kernel effect, not load balancing . . .	3
5.2. Once the kernel is held constant, load balancing is a net loss	4
5.3. The apparent “OpenCL regression” was the integrated GPU	4
5.4. GPU run-to-run variance is $\sim 10\text{--}25\%$	4
5.5. OpenCL work-group limit at block 512	4
6. From CPU+GPU to iGPU+dGPU	5
7. Conclusion: does static load balancing pay off here?	5

Índice de Figuras

1. Hybrid scaling. Left (log-log, all backends): the GPU backends sit $\sim 160\text{--}230\times$ above the flat CPU line; the anomalous OpenCL curve hugs ~ 1.7 Gcells/s. Right (GPU band): CUDA (<code>life_global</code>) peaks early then sags; the halo kernel holds ~ 29 Gcells/s at large N ; the auto-split hybrid climbs from far below toward the GPU band as the per-step overhead amortizes, only catching up near $N = 16384$	3
---	---

Índice de Tablas

1. Best throughput per backend per grid size N (Gcells/s, kernel time). The last column, “GPU halo-only”, is the hybrid’s GPU path run with <code>-cpu-frac 0</code> (halo kernel, full board, no CPU) — a control that isolates the kernel from the load balancing.	1
2. Calibrated rates and the resulting split. R in Mcells/s. The split is stable at 0.6% CPU at every size, exactly as $R_{\text{cpu}}/R_{\text{gpu}} \approx 0.006$ predicts.	2
3. Like-for-like at block 128, mean of repeated runs (Gcells/s). The halo kernel beats <code>life_global</code> with zero CPU; adding the CPU and the coordination (hybrid auto) gives it all back.	3

1. Scope and setup

Backend under test: the `HybridEngine` (branch `static_load_balancing`) — CPU + GPU on one board, split by rows, the partition chosen by DLT. Hardware: an NVIDIA GTX 1660 Ti Max-Q (`sm_75`, 60 W power ceiling) and a 12-thread CPU, on Linux. Data: `results/linux/sweep_{gpu_c}` from `scripts/sweep.sh`; figure `img/hybrid_load_balancing.png`.

- Metric. Cells evaluated per second, $\text{Mcells/s} = (\text{rows} \times \text{cols} \times \text{gens}) / (t \times 10^6)$, measured with the `NullRenderer` so host $\leftrightarrow$ device transfers and drawing never enter the timed loop. Timing is in-program and uniform per backend (CPU and hybrid: `std::chrono`; CUDA: `cudaEvent`). The hybrid figure is the wall of the concurrent CPU+GPU region, i.e. $\approx$ the slower of the two nodes.
- Sweep. Experiment A is the GPU block $\times$ shared/local sweep; experiment B is the scaling vs. N for CPU(threads) / CUDA / OpenCL / hybrid. We report the best (highest-Mcells) row per configuration, matching “mejor tiempo” in the graded report. Bounded edges, deterministic random fill (`-seed 1`: identical work across backends).
- Controlled follow-ups (this analysis). Single-knob runs that isolate why the hybrid behaves as it does: pure CUDA vs. the halo kernel on the full board (`-cpu-frac 0`) vs. the auto-split hybrid, all at block 128 so the only variable is the code path.

GPU health was verified before and after the sweep (idle 4.9 W, 39–41 °C, 60 W ceiling, no power-cap creep, no throttle flags), so the GPU numbers are not thermally compromised. Run-to-run boost-clock variance is real, though — see [S5.4](#).

2. Throughput results

Tabla 1: Best throughput per backend per grid size N (Gcells/s, kernel time). The last column, “GPU halo-only”, is the hybrid’s GPU path run with `-cpu-frac 0` (halo kernel, full board, no CPU) — a control that isolates the kernel from the load balancing.

N	CPU (best t)	CUDA	OpenCL	Hybrid (auto)	GPU halo-only
512	0.151	24.76	1.67	5.20	14.57
1024	0.146	27.55	1.70	9.56	25.07
2048	0.144	31.52	1.70	19.91	30.70
4096	0.142	32.25	1.69	22.75	31.42
8192	0.147	26.20	1.69	25.12	28.79
16384	0.152	24.83	1.68	28.46	28.72

The headlines from the table:

- The CPU plateaus at ~ 0.15 Gcells/s regardless of N : a 1-byte 9-point stencil is memory-bandwidth bound, and beyond ~ 6 threads the cores contend for the same bandwidth (scaling is flat, not linear). This sets R_{cpu} .
- CUDA is $\sim 160\text{--}230\times$ the CPU, peaking ~ 32 Gcells/s at $N \approx 2048\text{--}4096$, then easing to ~ 25 at $N = 16384$ (larger working sets spill L2, so the kernel becomes more memory-bound).
- OpenCL came out anomalous (see $\mathcal{S}5.3$). At 1.7 Gcells/s it is $\sim 15\times$ below CUDA, which is not representative of this hardware, and is excluded from the conclusions.

3. The load-balancing split (DLT $\mathcal{S}11.3$)

The calibration phase times a few CPU-only and GPU-only steps on the actual grid and applies the all-finish-together optimum:

$$\text{part}_{\text{gpu}} = \frac{R_{\text{gpu}}}{R_{\text{cpu}} + R_{\text{gpu}}}, \quad \text{part}_{\text{cpu}} = 1 - \text{part}_{\text{gpu}}, \quad (1)$$

with $R = 1/p$ the measured throughput (cells/s), converted to a row split $s = \text{round}(\text{part}_{\text{cpu}} \cdot R)$.

Tabla 2: Calibrated rates and the resulting split. R in Mcells/s. The split is stable at 0.6 % CPU at every size, exactly as $R_{\text{cpu}}/R_{\text{gpu}} \approx 0.006$ predicts.

N	R_{cpu}	R_{gpu}	GPU/CPU	CPU rows	CPU share	DLT bound $R_{\text{cpu}}+R_{\text{gpu}}$
2048	157.6	24 504	$156\times$	13	0.6 %	24 661
4096	163.8	27 176	$166\times$	25	0.6 %	27 340
8192	158.5	25 271	$159\times$	51	0.6 %	25 430
16384	158.0	27 286	$173\times$	94	0.6 %	27 444

The split matches the $\mathcal{S}11.3$ optimum to the row, and the two nodes finish together: at $N = 16384$ the CPU needs $94 \times 16384/158$ Mcells/s ≈ 9.74 ms and the GPU needs $16\,290 \times 16384/27\,286$ Mcells/s ≈ 9.78 ms. The DLT machinery works precisely as the book describes — this is not a tuning failure.

The catch is the ceiling. Because the CPU’s optimal share is only 0.6 %, the DLT “equivalent node” throughput $R_{\text{cpu}} + R_{\text{gpu}}$ is at most 0.6 % above the GPU alone. That is the entire prize, and it is tiny.

4. The figure

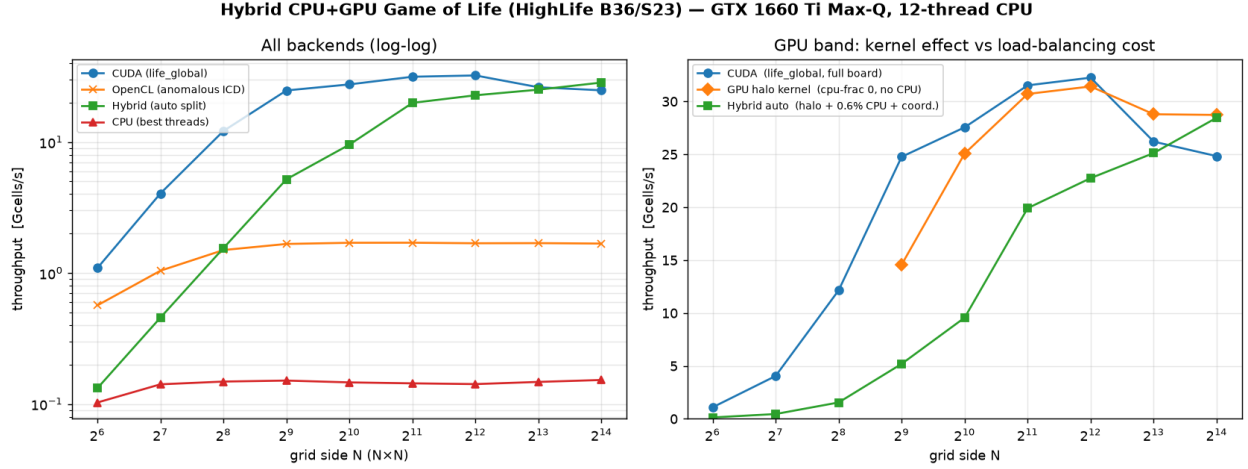


Figure 1: Hybrid scaling. Left (log-log, all backends): the GPU backends sit $\sim 160\text{--}230\times$ above the flat CPU line; the anomalous OpenCL curve hugs ~ 1.7 Gcells/s. Right (GPU band): CUDA (`life_global`) peaks early then sags; the halo kernel holds ~ 29 Gcells/s at large N ; the auto-split hybrid climbs from far below toward the GPU band as the per-step overhead amortizes, only catching up near $N = 16384$.

5. Unexpected findings

5.1. “Hybrid beats CUDA at large N ” is a kernel effect, not load balancing

The raw sweep shows the hybrid mean exceeding the CUDA mean at $N = 16384$ (27.4 vs. 24.6 Gcells/s, +11%) with low variance — too consistent to be noise, yet impossible to attribute to the CPU, whose DLT-optimal contribution is only 0.6%. A controlled run resolves it (block 128, full board):

Tabla 3: Like-for-like at block 128, mean of repeated runs (Gcells/s). The halo kernel beats `life_global` with zero CPU; adding the CPU and the coordination (hybrid auto) gives it all back.

N	CUDA <code>life_global</code>	Halo kernel (<code>-cpu-frac 0</code> , no CPU)	Hybrid (auto)
4096	25.68	26.61 (+3.6%)	18.25 (−29%)
16384	25.14	28.12 (+12%)	25.39 (+1%)

The halo kernel is intrinsically faster than `life_global`, with zero CPU involvement. The

reason: `life_global` does a per-cell vertical bounds check (if `(nr < 0 || nr >= rows)`, or a modulo under `-wrap`) for every neighbour row; the halo kernel reads its vertical neighbours unconditionally from always-present ghost rows, so it is branch-free in the vertical direction. At large N the kernel is compute/branch-bound (not launch-bound), so removing those branches buys 10–16 %. The “hybrid beats CUDA” headline was the hybrid’s GPU slice quietly running a better kernel — a confound, not a balancing win.

5.2. Once the kernel is held constant, load balancing is a net loss

Comparing like-for-like (both on the halo kernel), hybrid-auto is below halo-only at every size — -29% at $N = 4096$, $\approx$ break-even at $N = 16384$. The per-step coordination (ghost-row exchange + waking the worker pool + synchronising the two nodes, $\sim 0.3\text{--}1\text{ ms/step}$) costs more than the 0.6% the CPU offloads. Concretely at $N = 16384$: the DLT-optimal split saves the GPU $\sim 0.6\%$ of its time ($\sim 0.06\text{ ms}$) by handing 94 rows to the CPU, while the coordination adds $\sim 0.6\text{--}1\text{ ms}$ — a clear net loss that only looks like break-even because the halo kernel’s own speed-up cancels it.

5.3. The apparent “OpenCL regression” was the integrated GPU

This box previously measured OpenCL at $\sim 37\text{ Gcells/s}$, on par with CUDA (both lower through the same PTX/SASS on NVIDIA). A later sweep measured 1.7 Gcells/s and was first recorded as a driver/ICD regression. It was not. With explicit device pinning (the `GOL_OCL_DEVICE` environment variable, added in `OclDeviceSelect.hpp`), the 1.7 Gcells/s figure is the Intel UHD 630 integrated GPU, which the default device selection picks because it enumerates as OpenCL platform 0. Pinned to NVIDIA, OpenCL runs at $\sim 23\text{--}28\text{ Gcells/s}$, matching CUDA. So there was no regression and no reboot is needed — only a silent device choice. This reframes the exercise: the “slow OpenCL” was a second, real GPU sitting idle, which motivates the two-GPU split of $\mathcal{S}6$.

5.4. GPU run-to-run variance is $\sim 10\text{--}25\%$

A single CUDA run at $N = 4096$ gave 24.9 Gcells/s vs. the best-of-5 of 31.8 — boost-clock and scheduling variance, largest at small N (per-rep CV up to $\sim 30\%$ at $N = 2048$, down to $\sim 1\text{--}5\%$ at $N = 16384$). Any margin smaller than this band — including the entire 0.6% load-balancing prize — is unmeasurable. Report best-of- N and treat sub- 10% differences at small grids as noise.

5.5. OpenCL work-group limit at block 512

`-block 512` fails OpenCL with `CL_INVALID_WORK_GROUP_SIZE` (code -54) on this device (CUDA runs 512 fine). The sweep skips it gracefully; it is a device limit, not a

regression. Block 64–128 is optimal on both engines anyway.

6. From CPU+GPU to iGPU+dGPU

The negligible-CPU verdict says the CPU is the wrong second node; the corrected OpenCL finding (§5.3) supplies a better one — the Intel UHD 630 integrated GPU, reachable only through OpenCL. The engine was generalized from a fixed [CPU, GPU] pair to an ordered list of nodes, each a CPU pool, a CUDA discrete-GPU partition, or an OpenCL partition pinned to a chosen device, split by the same DLT rule $part_i = R_i / \sum_j R_j$. The default composition is iGPU + dGPU, with the CPU off by default. Adjacent nodes exchange the same one-row ghost halo; inter-GPU seams stage through host memory, as there is no cross-vendor peer copy. Every composition (`igpu,dgpu`, `cpu,dgpu`, the three-way `cpu,igpu,dgpu`) still verifies bit-for-bit against the CPU oracle in both edge modes.

The regime is more balanced than CPU+GPU, but still one-sided. Calibrated on this box, $R_{\text{iGPU}} \approx 2.1$ Gcells/s against $R_{\text{dGPU}} \approx 28$ Gcells/s (CUDA), so the DLT-optimal iGPU share is $\sim 7\%$ — an order of magnitude larger than the CPU’s 0.6%, but still small. At $N = 8192$ the auto-calibrated iGPU + dGPU hybrid reaches ~ 27 Gcells/s against pure dGPU’s ~ 28 : a few percent below, never above. The per-step host-staged seam exchange — two device-to-host reads and two host-to-device writes of a C -byte row — plus the integrated GPU’s higher launch and synchronization latency slightly exceed the $\sim 7\%$ of compute the iGPU adds.

Two practical notes. The split is sensitive to calibration length: ten steps under-measure the discrete GPU because its boost clocks have not ramped, so the DLT split hands the weak iGPU too many rows and the iGPU becomes the bottleneck; `-calib-steps 50` nearly closes the gap. And the run-to-run variance (§5.4) is comparable to the $\sim 7\%$ at stake, so the sub-dGPU margin sits at the edge of measurability. The verdict matches the CPU+GPU case: the DLT machinery is correct, but on a node pair this lopsided the communication cost eats the minority node’s contribution. A two-GPU win would need devices within a small factor of each other, or a decomposition whose seam does not cross the host boundary every step.

7. Conclusion: does static load balancing pay off here?

The implementation is correct and the theory is vindicated, but the regime is wrong for a win.

- **Correctness.** The hybrid verifies bit-for-bit against the `CpuEngine` oracle in both edge modes and across all splits; the calibrated partition matches the §11.3 optimum to the row; the two nodes finish together. As a teaching realization of DLT on a genuinely heterogeneous CPU+GPU pair (disjoint memory, real seam communication), it does exactly what the book says.

- But this is the textbook “one node dominates” regime. With the GPU $\sim 160\times$ the CPU, the DLT-optimal CPU share is 0.6%, so the theoretical ceiling over pure GPU is $\sim 0.6\%$ — and the per-step ghost exchange + pool wakeup + sync exceed that. The hybrid ranges from -29% (small/mid N , overhead dominated) to break-even (large N , overhead amortized). It never meaningfully beats pure GPU.
- The same holds one step up the device hierarchy. Replacing the CPU with the idle integrated GPU ($\mathcal{S}6$) raises the minority share from 0.6% to $\sim 7\%$, but the host-staged inter-GPU seam still costs more than that, so iGPU + dGPU lands a few percent below pure dGPU — more balanced, same verdict.
- The only speed-up in the data is a kernel-engineering side effect. The ghost-cell halo kernel, built to make domain decomposition possible, is incidentally 10–16% faster than the original global kernel at large N because it trades vertical boundary branches for unconditional ghost-row reads.

Recommendations

1. Ship pure GPU with the halo (branch-free) kernel. That is the real lever this exercise surfaced — a free 10–16% at large N , no CPU, no coordination. Consider promoting the halo formulation to the default global kernel (it needs only a one-row zero-pad at the board’s top/bottom under bounded edges).
2. Keep the hybrid as a correct DLT reference, not a production path on this hardware. It becomes a win only when R_{cpu} and R_{gpu} are within a small factor (say $< 5\times$): a vectorized / bit-sliced CPU kernel (raising R_{cpu} by 1–2 orders of magnitude), a weaker / shared GPU, or an integrated GPU + CPU. There the offloaded share is large enough to dwarf the fixed coordination cost.
3. Re-run OpenCL after a reboot before regenerating committed OpenCL figures; the current 1.7 Gcells/s is a transient ICD pathology, not this hardware’s OpenCL performance.

The bottom line: static load balancing is the right tool for balanced heterogeneous nodes, and the implementation proves it can be done correctly and cheaply. On a GTX 1660 Ti paired with a scalar CPU stencil it has nothing to balance — the honest result is “ $\approx$ pure GPU”, and the interesting performance story is the kernel, not the split.